RELATIONAL DATABASES

Intro to Databases

Database: An organized collection of electronic information that's accessed via computing devices.

Relational database: A database model made up of interconnected tables with these parts:

- **Attribute:** A column in a table.
- **Record:** A row in a table.
- **Primary key:** A unique ID for each record.
- **Data value:** A single piece of data.

Structured Query Language (SQL): A programming language used to create, manage, and manipulate a database.

Query: A request for information from a database.

View: A filtered set of data formatted for display.

Stored procedure: A pre-compiled SQL statement that helps automate routine tasks.

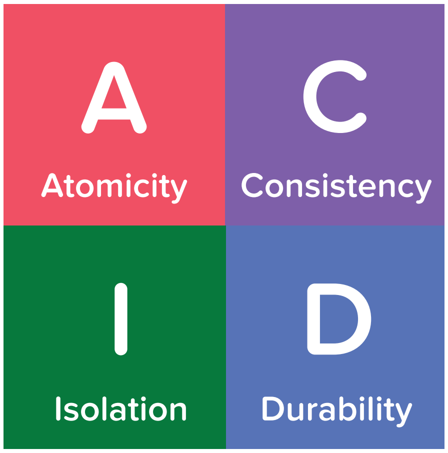
Index: A data structure that labels data to speed up queries.

Transaction: A set of database operations that work together as a unit.

To maintain the data's quality and reliability, a database must be ACID compliant.

- Atomicity: Multi-part transactions don't succeed unless all parts succeed.
- Consistency: The database is correct before and after every transaction.
- Isolation: Concurrent transactions are kept separate to prevent errors.
- Durability: Once data is written, it becomes permanent to prevent loss.

Non-relational database: A database model with greater flexibility to handle unstructured data, change the database schema, deal with incomplete data, and scale in size.



Non-relational databases are also called Not Only SQL or NoSQL databases because they don't necessarily use SQL as their primary query language.

Database Design

Database design follows these six steps:

1. Define mission & objectives
2. Analyze data requirements
3. Create an ER Diagram
4. Create a relational schema
5. Normalize the schema
6. Check your work

Entity-Relationship Diagram (ER Diagram): A detailed roadmap showing the data being collected and the relationships between data.

Entities: Objects and events from the real-world context, which will become tables in the database.

Attributes: Characteristics of entities, which will become columns in the tables.

One-to-many relationship: One object of type A is related to many objects of type B.

Many-to-many relationship: An object of type A is related to many objects of type B, and an object of type B is also related to many objects of type A.

One-to-one relationship: An object of type A relates to just one object of type B.

Business rules: Organization-specific rules for data.

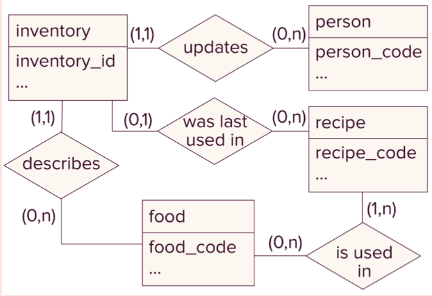
Data dictionary: A compilation of a database's business rules, entities, attributes, relationships, data types, constraints, and other specifications.

Relational schema: A model that shows how tables will be built and connected in the database.

Junction table: A solution to redundant data found in many-to-many relationships. It adds an additional table to create multiple one-to-many relationships.

Foreign keys: Attributes that reference the primary key of another table.

Sample ER Diagram



In an ER Diagram, ordered pairs denote the minimum and maximum of each object involved in a relationship.

Primary Key Criteria

- Unique
- Not null
- Not reliant on sensitive personal info
- Not modified often (especially not by end users).

Sample Relational Schema

person (person_code, first_name, last_name, birth_date, age)

food (food_code, food_name, calorie_count, category, unit_price)

inventory (inventory_id, food_code, food_quantity, inventory_status, expiration_date, purchased_by, purchased_date, last_updated_date, last_updated_by_code, last_updated_reason, last_recipe_code)

recipe (recipe_code, recipe_name, total_time, serving_size, link)

ingredient (ingredient_id, recipe_code, food_code, unit_quantity)

Data integrity: The accuracy, completeness, and consistency of the information in a database.

Normalization: The process of ensuring data integrity.

Normalization includes has three goals:

1. Maintain consistency.
2. Prevent data redundancy, or multiple occurrences of the same piece of data.
3. Ensure proper data dependencies, or relationships between pieces of data.

First normal form (1NF): Data follows these rules:

- Each column name is unique.
- Each data value only contains one piece of data.
- Each column contains data in one format.

Second normal form (2NF): Eliminates partial dependencies, which is when a table with a composite primary key has a column that only depends on part of that primary key.

Third normal form (3NF): Requires tables to avoid transitive dependencies, in which one column depends on another column more than it depends on the primary key.

Building a Database

Constraints specify rules for data in a column, which help enforce an organization's business rules.

Sample syntax:

```
CREATE TABLE tablename (
    column1 DATATYPE CONSTRAINT,
    column2 DATATYPE CONSTRAINT,
);
```

```
INSERT INTO table_name (column1, column2)
VALUES (value1, value2);
```

```
ALTER TABLE table_name
ADD PRIMARY KEY (column_name);
```

Types of Constraints

- `CHECK` evaluates each value with a Boolean expression.
- `UNIQUE` ensures that there are no repeating values in a column.
- `NOT NULL` ensures that a record can't point to a NULL value.
- `PRIMARY KEY`
- `FOREIGN KEY`

Remember: you can also import data from a CSV file!

Modifying a Database

Trigger: An action or stored procedure that runs when a particular condition occurs.

Conditions include:

- `ON DELETE`: When a record is deleted.
- `ON UPDATE`: When a record is updated.
- `ON CONFLICT (<conflict_target>)`: When there's a conflict with a particular column.

Actions include:

- `SET NULL`: Makes every reference to the record `NULL`.
- `CASCADE`: Applies a change to every associated record.
- `DO NOTHING`: Won't make any changes.
- `DO UPDATE SET`: Updates the existing data.

Sample syntax:

```
ALTER TABLE <source_table>
ADD CONSTRAINT "<constraint_name>"
FOREIGN KEY (<source_attribute>)
REFERENCES <target_table> (<target_attribute>);
```

```
ALTER TABLE <table_name>
DROP COLUMN <column_name>;
```

```
UPDATE <table_name>
SET <column_name> = <expression>;
```

Example with `UPDATE SET`

```
ON CONFLICT (recipe_code)
DO UPDATE
SET recipe_name =
EXCLUDED.recipe_name
```

Queries

Aliases use `AS` to temporarily translate names in the database into something more business friendly when the output, or **result set**, is presented on the screen.

`WHERE` specifies which records to return and which to exclude.

Wildcard characters: Stand-ins for other characters, represented by `%` and `_`.

Basic Query Syntax

```
SELECT <column_name> FROM
<table_name>;
```

Aggregate functions: Functions that take many values and return just one.

`GROUP BY` groups result sets according to the attribute specified in the query.

`HAVING` filters grouped data.

SQL's Order of Operations

1. `FROM` `JOIN`
2. `WHERE`
3. `GROUP BY`
4. `HAVING`
5. `SELECT`
6. `ORDER BY` `LIMIT`

Joins and CTEs

Inner join: Only returns records that appear in both tables.

Left join: Returns all records in the first table and any in both tables.

Right join: Returns all records in the second table and any in both tables.

Outer join: Returns all records in the first table, all in the second, and any in both tables.

The syntax for a join:

```
SELECT table1.col, table2.col
FROM table1
JOIN table2
ON table1.matching_col = table2.matching_col;
```

Subquery: A query surrounded by parentheses and nested inside another query.

Common table expression (CTE): A query that names a result set, which can then be called inside another query.

The syntax for a CTE:

```
WITH <CTE_name> AS (<query>)
```

Multiple CTEs are separated by commas, and the second CTE only needs the `AS` keyword.

Formatting Result Sets and Writing Efficient Queries

`ORDER BY` determines the order that results are presented in.

`LIMIT` determines how many records are returned in the result set.

Database context: The connection at the top of a query window specifying the database.

Record locking tools prevent two transactions from modifying or deleting the same record.

Queries can be saved as stored procedures.

Result sets can be exported as CSV files to present to stakeholders.